configuring the code-base are:

- CMAKE_INSTALL_PREFIX: The location where the MySQL server will be installed.
- COMMUNITY_BUILD: Enables/disables the community features in MySQL.
- ENABLED_PROFILING: Enables/disables query profiling code.
- MYSQL_DATADIR: This is the default directory where data will be stored. It can be set in the configuration file as well, after installation.
- WITH_INNOBASE_STORAGE_ENGINE: Enables/disables InnoDB, a transactional storage engine.
- WITH_DEBUG: Enables/disables debugging capability; this increases the binary size, but can be very useful while getting support from forums, etc, in case your installation fails for some reason. A related option is ENABLE_DEBUG_SYNC.
- WITH_PARTITION_STORAGE_ENGINE: Enables/disables the partition storage engine.
- WITH_SSL: Enables/disables SSL support in the server and client. Enable this if you see any need for replication in the future; unencrypted replication is a risk.
- CMAKE_CXX_FLAGS: Sets the C++ compiler flags *-O3 -march=native -mtune=native -msse -msse2 -mmmx* (provided you have a CPU with a decent cache size—else use *-Os instead of -O3).*
- CMAKE_C_FLAGS: Sets the C compiler flags. MySQL has no C code, so this is not required. You might want to set it to the same value as CXXFLAGS, for sanity.
- DISABLE_SHARED: Builds MySQL statically, with all plug-ins compiled in. This is not preferred; it will unnecessarily consume memory for unwanted features/plug-ins.

I have described the most important options, all of which are enabled by default. If you want a look at many more, use *ccmake or cmake-gui*, or refer to the MySQL documentation at *http://j.mp/rlVYri*.

Server configuration

MySQL's single configuration file, *my.cnf*, is located in *CMAKE_INSTALL_PREFIX/etc* or *MYSQL_DATADIR* (deprecated). You can also pass the location of the configuration file to *mysqld* while starting it. Some of the configuration options in *my.cnf* in the *[mysqld]* section are:

- *bind-address:* The address on which the server will listen for connections. If you're only going to connect from the local host, don't use this; instead, use the *skip-networking* option, as a security measure.
- *user:* The account as which *mysqld* will run; again, a security measure. This user must have R/W access to *MYSQL_DATADIR.*
- *datadir:* By default, the compile-time path in *MYSQL_DATADIR*. Change it if you want to relocate the data directory.
- *character-set-server:* Keep as UTF8 (Unicode), unless you have special reason to change it.
- *character-set-client:* Same as the previous option, but this will tell the client to use the specified character set.
- *default-character-set:* The default set to use while creating tables.
- *default-storage-engine:* This default (MyISAM) will be used if an engine is not specified in the *CREATE* statement.
- *skip-innodb:* Disable the InnoDB storage engine. If you statically compiled MySQL, you need to use *ignore-builtin-innodb*.
- *key_buffer:* The MyISAM key buffer; should be kept as large as possible (of course, sparing DRAM for other applications and services). The larger it is, the better the performance. On a dedicated MySQL server, it is usually at least a quarter of the total memory (and not more than half). It should be enough to hold all the MyISAM indexes (.MYI files) in memory. This can be checked via variables, which can be obtained by the *SHOW VARIABLES* command at the MySQL command prompt, or using tools like phpMyAdmin. Obtain these variable values, and divide *key_read* by *key_read_requests*; the value should be < 0.01. And if you divide *key_write* by *key_write_requests*, the value should be < 1.
- *table-cache:* Every time MySQL opens a table, it saves it in cache, to speed up access. The variable *opened_tables* in server status will tell you what you need to set this to; some use cases have seen values as high as 20000. Keep *opened_tables* as low as possible.
- *read_buffer_size:* The default is 128 K, and usually this is set to 1M or 2M. The performance depends on CPU cache, disk speed and other factors; you should run a test in your environment, as described at *http://j.mp/nV3SJ5*.
- *query_cache_type:* The value 0 means the query cache is disabled; 1 implies that the query cache is enabled for all queries that don't have *SQL_NO_CACHE* specified in the query; and 2 means that no query is cached unless *SQL_CACHE* is specified in the query.
- *query_cache_size:* The larger the better, but also check the available DRAM.
- *query_cache_limit:* The maximum size of the result set that will be stored in the query cache. This should be large if you have queries producing large result sets and have a lot of DRAM to spare. Watch the variables